

Ream: Sparse, Address-Explicit Spreadsheet Serialization for Language Models

Siqi Chen

Runway Financial, Inc.

1 Abstract

Large language models do not read spreadsheets directly. They read text produced by a serializer, and that representation choice can either preserve or erase the cues the model needs to reason correctly. Prior work on table serialization has focused mostly on flat tables, with limited treatment of multi-sheet workbooks, sparse layouts, formulas, defined names, and structured tables. This paper introduces **Ream**, a UTF-8, line-oriented, sparse text format for spreadsheet comprehension. Ream assigns each emitted row its absolute worksheet row number, uses A1-style selectors outside formulas, preserves formula text in R1C1 notation, supports defined names and named tables, and omits blank cells by default.

We evaluate Ream against 10 alternative serialization formats spanning delimiter-based, markup-based, record-based, structured, compressed, and native XML baselines. The reported benchmark covers three corpora—enterprise multi-sheet workbooks from FRTR-Bench, real single-sheet worksheets from MiMoTable, and formula-derived QA tasks from NL2Formula—across three OpenAI models: GPT-4o-mini, GPT-5.4-nano, and GPT-5.4. The study comprises more than 19,000 completed model calls under a fixed prompting and scoring protocol. On the enterprise workbook benchmark, Ream reaches 92.0% accuracy on GPT-5.4, tying the strongest reported baseline while using 26% fewer tokens than Cell-Address Markdown and 53% fewer than JSON. On GPT-5.4-nano, Ream attains the best reported FRTR accuracy at 78.5%. Across the three reported corpora, Ream lies on the accuracy–cost Pareto frontier: it is never far from the most accurate format on small tables and remains strong when workbook scale and context pressure increase.

Two conclusions follow. First, serialization format is a first-order systems decision for complex spreadsheets: on enterprise workbooks the spread between the strongest and weakest reported formats exceeds 30 percentage points on GPT-5.4 and 50 points on GPT-5.4-nano. Second, the right claim is not that one format dominates everywhere. On small single-sheet tables, several value-preserving formats perform similarly. Ream’s advantage appears when a workload is wide, sparse, multi-sheet, or close to model context limits. The current benchmark evaluates question answering over spreadsheet values rather than formula generation, so Ream’s formula, name, and table support should be understood as representational capabilities whose full downstream value remains to be measured.

2 Introduction

Spreadsheets remain the dominant programming environment for business analysis. Forecasting, planning, reporting, operational review, and scenario modeling are still built in workbook form.

When language models are asked to answer questions about those workbooks, however, the model does not consume `.xlsx` directly. It consumes a textual representation created by a serializer. That representation determines whether the model sees sheet boundaries, row identity, sparse layout, repeated formulas, names, and tables—or a flattened approximation that discards those signals.

That representation layer is often treated as an implementation detail. In practice it is not. A delimiter-based export such as CSV may be compact for dense single-sheet tables but loses multi-sheet structure, formulas, and named objects. JSON preserves explicit keys but adds heavy syntactic overhead. HTML and XML preserve hierarchy but are costly in tokens and require the model to look through markup. Cell-addressed formats preserve location but may become verbose when every cell is emitted individually. The choice of serialization is therefore part of the reasoning system, not just its plumbing.

Existing work has not fully addressed this gap. Prior studies on table serialization focus mainly on flat tables [6, 3]. Spreadsheet-specific work has proposed compressed encodings and address-aware representations [1], while spreadsheet benchmarks have concentrated on task difficulty rather than serialization as an independent variable [2, 4, 5, 7]. Multi-sheet enterprise workbooks bring a different set of requirements: sparse structure, large row counts, formula bands, cross-sheet references, defined names, tables, and strict context budgets.

This paper studies that setting through the lens of representation design. We introduce **Ream**, a sparse text format built around five goals:

1. **Address fidelity.** Every emitted cell should retain a stable worksheet identity.
2. **Sparse efficiency.** Blank background should not dominate token count.
3. **Formula locality.** Formula text should preserve copy-fill semantics without requiring the serializer to enumerate each cell separately.
4. **Workbook structure.** Sheet boundaries, named ranges, and tables should survive serialization.
5. **Deterministic roundtripping.** A serializer and parser should map between workbook state and text without location drift.

We then benchmark Ream against 10 alternative formats on three reported corpora and three model sizes. The results support a narrow but useful claim: **Ream is not universally best, but it is consistently close to the best accuracy while using materially fewer tokens on complex workbooks.** On enterprise spreadsheets, where token cost and navigation errors matter, that trade-off is strong enough to change practical system design.

The paper makes three contributions:

1. It defines Ream as a spreadsheet-oriented text representation that combines sparse row encoding, A1 selectors, R1C1 formulas, and named-object support.
2. It presents a broad serialization benchmark covering 11 formats, 3 model sizes, and more than 19,000 completed evaluations.
3. It shows that representation choice matters sharply for complex workbooks and much less for small single-sheet tables, which helps narrow the conditions under which serialization work is worth doing.

3 Problem setting and design objectives

3.1 Workbook serialization as a modeling problem

Let W denote a workbook containing one or more sheets, where each sheet is a sparse mapping from absolute cell coordinates to typed cell contents. A serializer S maps W to a text sequence $T = S(W)$. An LLM-based spreadsheet system then answers a question q by applying a model M to the pair (q, T) .

The serializer is part of the model’s effective input distribution. A poor serializer can force the model to recover row identity by counting delimiters, infer sheet boundaries from incidental text, or infer formula copy semantics from repeated strings. A better serializer exposes those invariants directly.

A spreadsheet serializer intended for LLM reasoning should satisfy five design objectives.

D1. Unambiguous addressability. A cell referred to in text should correspond to a unique worksheet location.

D2. Sparse compactness. The cost of blank cells should be close to zero. This matters because workbooks are often sparse even when their nominal used range is large.

D3. Relative-formula preservation. Formula bands are easier to compress and interpret when relative references are preserved explicitly.

D4. Structural coverage. Multi-sheet boundaries, names, and tables should be available to the model because spreadsheet authors rely on them semantically.

D5. Deterministic normalization. The format should admit a canonical form so that parse-serialize cycles are stable and diffs are meaningful.

Ream is designed for this objective set, not for full Excel fidelity. It is a representation for comprehension and interoperable editing of scalar spreadsheet content, not an archival replacement for OOXML.

3.2 Why existing generic formats are insufficient

Dense row-major formats such as CSV and Markdown are simple and token-efficient for small dense tables, but they force the model to recover absolute location from position and typically discard workbook structure. Hierarchical formats such as HTML, XML, and JSON preserve more structure, but their syntactic scaffolding becomes expensive on large workbooks. Cell-addressed formats preserve location, but naïve forms emit one address per cell and therefore lose the benefits of sparsity. Native OOXML retains the entire workbook, but it exposes the model to implementation-level XML rather than analytic structure.

The design question is therefore not whether an LLM can parse any one of these formats at all. It is whether the representation preserves the right cues at the lowest possible token cost for the kinds of workbooks analysts actually use.

4 The Ream format

4.1 Core ideas

Ream is governed by six rules.

1. Each emitted worksheet row begins with its absolute row number as a plain line label.
2. Columns advance by position, with optional A1-style prefixes such as B= or B:M= to jump over blank regions or compact repeated runs.
3. Selectors outside formulas use A1 notation.
4. Direct cell and range references inside formulas use R1C1 notation.
5. Defined names and structured table references are first-class formula operands.
6. Omitted cells are blank.

The format is line-oriented and UTF-8 text. A workbook contains sheet directives, optional name and table directives, row records, and optional metadata directives such as `#!HEADERS`, `#!FMT`, `#!TAG`, and `#!NOTE`. The underlying RFC-style specification defines canonical serialization, parser behavior, and extension points.

4.2 Example

The following example shows two sheets, a workbook-scoped name, row-level sparsity, R1C1 formulas, and metadata.

```
#!REAM 9
#!NAME tax_rate 'Assumptions'!B2

#!SHEET Assumptions
2 | Tax Rate | 0.25 |

#!SHEET "P&L"
#!HEADERS 1:1
#!NAME revenue_inputs B2:M2

1 | B=Jan | Feb | Mar | Apr | May | Jun | Jul | Aug | Sep | Oct | Nov | Dec |
2 | Revenue      | 1000 | 1050 | 1100 | 1150 | 1200 | G:M==RC[-1]*1.08 |
3 | COGS          | B:M==R[-1]C*0.6 |
4 | Gross Profit  | B:M==R[-2]C-R[-1]C |
6 | Headcount     | B:C=50 | D:M=52 |
7 | Avg Salary    | B:M=8500 |
8 | Payroll       | B:M==R[-2]C*R[-1]C |
9 | Rent          | B:M=15000 |
10 | Total OpEx    | B:M==R[-2]C+R[-1]C |
12 | Tax           | B:M==R4C*tax_rate |
13 | Net Income    | B:M==R4C-R10C-R[-1]C |

#!FMT 2:4 usd0
#!TAG B2:F2 "sales input"
#!TAG B13:M13 output
```

This example illustrates the separation of concerns that motivates Ream. External selectors and metadata use familiar A1 notation. Internal formula references use R1C1 so that the same formula string can be applied across a range while preserving relative semantics.

4.3 Why A1 outside formulas and R1C1 inside formulas

This split is deliberate.

A1 notation is the address language most spreadsheet users and most language models have already seen for sheet navigation and selection. It works well for directives, selectors, and sparse in-row jumps. R1C1 notation, by contrast, is better for repeated formulas because relative offsets are explicit. A band such as `B:M==R[-1]C*0.6` states that every cell in columns B through M uses the same relative relation to the row above. A1 notation can express that pattern only by repeating or by relying on implied fill behavior outside the text itself.

4.4 Formal properties

Ream is motivated by empirical performance, but several useful properties follow directly from the syntax.

Property 1 (address determinism). Within a sheet, each explicit cell assignment in canonical Ream maps to a unique absolute worksheet coordinate. This follows from two facts: each row record begins with an absolute row number, and the column cursor within the row is either advanced by position or reset explicitly by an A1-style prefix.

Property 2 (blank omission). A serializer need not emit blank cells. Any cell not addressed by a row record is blank by definition. This makes the representation naturally sparse.

Property 3 (copy-fill locality). A contiguous formula band can be represented by a single addressed range plus one R1C1 formula string. The serializer does not need to duplicate the formula text for every cell in the band.

Property 4 (canonicalization). Because directives, selectors, and row records have a canonical order, parse-serialize cycles can be made deterministic. This is useful for caching, diffing, and test fixtures.

These are simple properties, but they matter. Spreadsheet QA systems fail surprisingly often on row counting, sheet disambiguation, and formula locality, especially once the workbook stops looking like a small table.

4.5 Complexity intuition

Consider a worksheet row with width W , k non-blank cells, and r maximal emitted runs of identical scalars or formulas. Ignoring cell payload and tokenizer quirks, the structural overhead of a dense delimiter format scales with $O(W)$ because every column boundary must be represented. Ream’s structural overhead scales with $O(r \log W + k)$ when the row is sparse and uses addressed jumps. In realistic business models, where $k \ll W$ and long blank gaps are common, that difference is large enough to matter in context-limited prompting.

The empirical benchmark reflects this pattern: the formats converge on small tables but diverge sharply on wide enterprise workbooks.

4.6 What Ream does not attempt to encode

Ream is intentionally narrower than OOXML. In core form it does not encode sheet extents, cached formula values, row heights, column widths, merged cells, chart objects, conditional formatting, hidden state, or macro payloads. That boundary is a strength, not a weakness, for the present task.

Spreadsheet comprehension by LLMs is often bottlenecked by location, sparsity, and structure, not by presentation metadata.

5 Experimental protocol

5.1 Research questions

The benchmark addresses three questions.

RQ1. How much does serialization format affect spreadsheet question answering on complex enterprise workbooks?

RQ2. Which formats offer the best accuracy–token trade-off rather than the best raw accuracy alone?

RQ3. Do representation effects become larger for smaller models and more complex spreadsheets?

5.2 Baseline formats

We compare Ream against 10 alternative formats spanning six broad families. Table 1 groups them by structural style.

Table 1. Serialization formats evaluated in the benchmark.

Format	Family	Cell identity surface	Multi-sheet support	Provenance
Ream	Sparse cell-addressed	A1 row/column addressing	<code>#!SHEET</code>	This work
CSV	Delimiter	Positional	Text header	Baseline
Markdown	Markup	Positional	Text header	Baseline
HTML	Markup	DOM structure	Heading tag	Prior work [6]
XML	Markup	Row attributes / tags	<code><sheet></code> tag	Prior work [6]
JSON	Structured	Column-letter keys	Nested object	Baseline
Pandas	Record	Row index + column labels	Section headers	<code>df.to_string()</code>
Markdown-KV	Record	Header-value pairs	Section headers	ImprovingAgents (2025)
Cell-Address Markdown	Cell-addressed	Explicit A1 coordinates	Section headers	SpreadsheetLLM-style baseline [1]
Reverse-Index Values	Compressed	Value-to-address index	JSON object	This work
Raw OOXML	Native XML	Spreadsheet implementation tags	Native files	Baseline

The comparison set intentionally includes both natural human-readable formats (CSV, Markdown),

explicit-key formats (JSON), hierarchical formats (HTML, XML), and spreadsheet-specific address-aware formats.

5.3 Corpora

The reported benchmark covers three corpora that span a useful complexity range.

FRTR-Bench enterprise workbooks [2]. Thirty real-world enterprise workbooks are used to generate 1,403 candidate QA pairs across 13 question types from sheets with at most 1,000 rows. Two hundred questions are sampled for evaluation. These workbooks are multi-sheet, formula-bearing, and large enough for context pressure to matter.

MiMoTable [4]. Two hundred lookup and comparison questions over real XLSX tables. These sheets are generally single-sheet and much smaller than FRTR-Bench.

NL2Formula [7]. Two hundred formula-derived QA pairs converted from JSON to XLSX. These are tiny, mostly single-sheet tables intended to probe lightweight spreadsheet reasoning.

Table 2 summarizes the reported corpora.

Table 2. Evaluation corpora.

Corpus	n	Workbook structure	Typical scale	Origin
FRTR-Bench	200	Multi-sheet enterprise workbooks	2–5 sheets, 200–1,000 rows	Programmatic QA over XLSX
MiMoTable	200	Single-sheet real tables	<100 rows	Human-authored XLSX
NL2Formula	200	Single-sheet formula-derived tables	~10 rows	JSON converted to XLSX

5.4 Models and scoring

All experiments are run at temperature 0 on three OpenAI models: GPT-4o-mini, GPT-5.4-nano, and GPT-5.4. The draft benchmark associates GPT-4o-mini and GPT-5.4-nano with 128K context windows and GPT-5.4 with a 1M window.

Answers are scored using the same rules across formats:

- numeric answers are correct within a 5% tolerance,
- string answers use case-insensitive containment matching,
- booleans are normalized to semantic equivalence.

The planned evaluation count is

$$11 \text{ formats} \times 3 \text{ models} \times 3 \text{ corpora} \times 200 \text{ questions} = 19,800$$

with more than 19,000 completed calls reported in the draft benchmark.

5.5 Reporting conventions and uncertainty

A peer-reviewable serialization paper should distinguish at least three quantities:

1. **Accuracy**: proportion of all evaluation items answered correctly.
2. **Context-failure rate**: proportion of items that fail because the serialized prompt exceeds the available context budget.
3. **Conditional answer accuracy**: accuracy restricted to answerable items when context failures are excluded.

The current benchmark tables mix overall accuracy and conditional accuracy in a few places, particularly when context failures occur. That limitation should be made explicit because conditional accuracy can look better than end-to-end task success. Where the original table marks a score with a dagger, it indicates accuracy over answerable questions only.

A second reporting issue is statistical uncertainty. For corpora with $n = 200$, a binomial 95% interval is roughly ± 4 –7 percentage points depending on the baseline. That means 0.3–2.0 point differences should not be overinterpreted. The strongest evidence in this paper comes from large effect sizes on FRTR-Bench and from accuracy–cost trade-offs, not from tiny average differences on small-table corpora.

6 Results

6.1 Enterprise multi-sheet workbooks

Table 3 reproduces the reported FRTR-Bench results. These are the most informative results in the study because they involve multi-sheet workbooks and enough scale for token costs and location cues to matter.

Table 3. FRTR-Bench accuracy (%) on 200 enterprise questions. Entries marked (cond.) denote conditional accuracy over answerable questions only, as reported in the benchmark table.

Format	GPT-5.4	GPT-5.4-nano	GPT-4o-mini	Context failures (reported)
Ream	92.0	78.5	69.0	0
Cell-Address	92.0	75.5	67.0	0
Markdown				
Markdown-KV	91.5	77.0	69.1	6
JSON	89.5	78.0	73.9 (cond.)	58
XML	89.4	73.0	63.7 (cond.)	109
Reverse-Index Values	85.5	66.5	65.0	0
HTML	84.5	68.0	64.6	19
CSV	84.0	66.5	63.0	0
Markdown	83.0	70.0	62.5	0
Pandas	80.0	67.0	63.8	1
Raw OOXML	61.6	24.8 (cond.)	32.8 (cond.)	136

Three points stand out.

First, Ream ties the best reported GPT-5.4 score at 92.0% while beating the nearest address-aware alternative on token cost. The benchmark text reports an average of 45K tokens for Ream on FRTR, versus 61K for Cell-Address Markdown, 96K for JSON, and 137K for XML. That means Ream achieves the same top-line enterprise accuracy as Cell-Address Markdown with about three quarters of the token budget, and it avoids the heavy structural tax of JSON and XML.

Second, the advantage grows on smaller models. On GPT-5.4-nano, Ream leads the field at 78.5%, 12 points above CSV and 8.5 points above plain Markdown. The spread between the strongest and weakest reported format is 53.7 points. That is too large to dismiss as noise or benchmark idiosyncrasy.

Third, context failures matter. Ream is the only top-performing format in Table 3 with zero reported context failures. JSON and XML remain competitive on answerable examples, but their token overhead creates many failures on GPT-4o-mini. A realistic production system cares about end-to-end success, not only conditional accuracy on the subset that fits.

6.2 Single-sheet tables

Table 4 aggregates the reported results across the three corpora for GPT-5.4 and GPT-5.4-nano.

Table 4. Accuracy (%) across the three reported corpora.

Format	FRTR	MiMo	NL2F	GPT-5.4	FRTR	MiMo	NL2F	GPT-5.4-nano
				macro avg.				macro avg.
Ream	92.0	66.5	78.5	79.0	78.5	61.5	72.5	70.8
JSON	89.5	67.5	79.0	78.7	78.0	60.5	77.5	72.0
XML	89.4	66.5	78.5	78.1	73.0	61.0	77.0	70.3
Cell-Address Mark- down	92.0	67.5	75.5	78.3	75.5	61.5	73.0	70.0
Markdown- KV	91.5	54.0	78.5	74.7	77.0	49.5	74.0	66.8
HTML	84.5	66.5	78.0	76.3	68.0	61.5	76.0	68.5
CSV	84.0	66.0	76.5	75.5	66.5	60.0	73.5	66.7
Markdown	83.0	66.5	76.5	75.3	70.0	64.0	75.0	69.7
Pandas	80.0	67.5	77.5	75.0	67.0	60.0	74.5	67.2
Reverse- Index Values	85.5	66.0	76.5	76.0	66.5	52.0	63.5	60.7
Raw OOXML	61.6	66.5	78.5	68.9	24.8	58.0	76.5	53.1

On MiMoTable and NL2Formula, the leading formats are tightly clustered. The original benchmark

notes a spread of about 2 points on MiMoTable and 3.5 points on NL2Formula. Those corpora are small enough that most value-preserving formats fit comfortably within context windows and retain enough structure for the model to answer straightforward lookup or short formula questions.

This is an important negative result. It means Ream should not be sold as a universal win. On small single-sheet tables, the representation problem is easy enough that many formats work. Ream’s main value appears in the regime that generic table serializers do not target well: large, sparse, multi-sheet workbooks.

6.3 Accuracy–cost trade-off

The clearest claim supported by the benchmark is not raw dominance but **Pareto efficiency**.

- On enterprise workbooks, Ream matches the best reported GPT-5.4 accuracy and leads on GPT-5.4-nano.
- It does so with fewer tokens than the strongest competing address-aware or structured formats.
- It avoids the severe context-failure behavior observed for heavier hierarchical formats on the smallest model.

That pattern is stronger than the paper’s earlier claim of the “highest cross-corpus accuracy.” The macro-average difference between Ream (79.0) and JSON (78.7) on GPT-5.4 is only 0.3 points and should not be treated as decisive. What *is* decisive is that Ream delivers near-best accuracy without paying JSON/XML-level token cost.

6.4 Why the gains appear

The results are consistent with three mechanisms.

Explicit location cues. Absolute row labels and optional A1-style addressed jumps reduce the need for the model to count delimiters across wide rows.

Sparse efficiency. Ream omits blank cells and compacts repeated runs. That matters on real business models, where the semantic density of the sheet is often low even when the nominal grid is large.

Workbook-level structure. `#!SHEET`, `#!NAME`, `#!TABLE`, and `#!HEADERS` expose sheet boundaries and semantic groupings directly instead of leaving them implicit.

These mechanisms are plausible from both the design and the empirical side. What the present benchmark does not yet isolate is the marginal contribution of each one. An ablation study would be the next step.

7 Discussion

7.1 What Ream is actually good for

The benchmark and the specification suggest that Ream is a good fit for:

- enterprise planning and reporting workbooks,
- financial models with wide time-based layouts,
- sparse worksheets with named ranges or tables,
- pipelines where context budget or inference cost matters,
- prompt-time workbook inspection by humans and LLMs.

It is a weaker fit for dense database-like exports, arbitrary Excel feature preservation, or workloads where the workbook is tiny enough that any value-preserving format succeeds.

7.2 Representation, not reasoning, is the intervention

This paper studies an upstream systems choice. It does not claim a new reasoning method, a new model, or a new benchmark task. That distinction matters. The gain comes from changing what the model sees before it reasons, not from changing the model itself. In that sense Ream is closer to tokenizer-aware systems design than to benchmark-specific prompt engineering.

7.3 What peer reviewers are likely to ask

Three reviewer questions are predictable.

Is Ream better because it is more familiar to the model, or simply because it is shorter?

The current results support both explanations to some extent, but do not separate them. A clean ablation would compare A1 selectors versus numeric selectors at matched token counts, and sparse addressing versus dense row-major formats with identical content.

Does formula support matter in practice? Ream supports formulas, names, and tables in the representation, but the reported benchmark is mostly value-centric. A stronger follow-on evaluation should include formula tracing, formula generation, and cross-sheet dependency tasks.

How robust are the results across model families? The paper currently reports only OpenAI models. That is enough to motivate the design, but not enough to establish model-family invariance. These are not fatal weaknesses. They define a clean next experimental agenda.

8 Limitations and threats to validity

A serious serialization paper should state its boundaries plainly.

L1. Model-family scope. All reported experiments use OpenAI models. The relative ranking may differ for Claude, Gemini, or open-weight models.

L2. Benchmark scope. The corpora span valuable workloads but do not cover every spreadsheet task. In particular, formula editing, chart interpretation, and heavily formatted sheets are out of scope.

L3. Formula capability is under-tested. Ream’s support for R1C1 formulas, names, and structured tables is a design feature, not yet an empirically isolated source of gain in the reported benchmark.

L4. Reporting asymmetry for context failures. Some legacy result tables report conditional accuracy over answerable items. Future versions should report unconditional success, context-failure rate, and conditional accuracy separately for every format/model pair.

L5. Statistical power. With $n = 200$ per corpus in two of the three settings, small differences should be treated cautiously. The paper should not claim superiority from sub-point average gaps.

L6. Full reproducibility depends on release artifacts. The exact prompts, serialization scripts, item-level predictions, and token counts should be released with the evaluation harness.

9 Reproducibility checklist

To make the work easy to review and build on, the release package should include:

1. the Ream specification version used in the benchmark,
2. exact serializer implementations for all 11 compared formats,
3. exact prompts and system instructions,
4. item-level predictions and raw model outputs,
5. per-format token counts before truncation,
6. unconditional and conditional accuracy tables,
7. the sampled evaluation subsets for each corpus,
8. scripts for rebuilding tables and confidence intervals from raw runs.

This checklist is not cosmetic. Serialization papers are unusually sensitive to seemingly small implementation choices, and reviewers will rightly ask whether those choices are reproduced faithfully.

10 Conclusion

Ream is a sparse, address-explicit representation for spreadsheet workbooks intended for language-model comprehension. It uses plain row labels, A1 selectors, R1C1 formulas, and named-object support to preserve the structural cues that dense table formats often lose. Across more than 19,000 reported evaluations, Ream performs strongly on the workload where serialization matters most: large, sparse, multi-sheet enterprise workbooks. It ties the best reported GPT-5.4 accuracy on FRTR-Bench, leads on GPT-5.4-nano, uses materially fewer tokens than the closest high-performing competitors, and avoids the context-failure behavior that affects heavier structured formats.

The paper’s strongest claim is therefore not that Ream wins every benchmark. It does not. The stronger and more defensible claim is that Ream occupies an attractive point on the accuracy–cost frontier and that serialization format becomes a first-order design choice once spreadsheets are large enough to stress model context and navigation. That is a useful result on its own, and it opens a concrete research agenda around ablations, model-family robustness, and formula-centric tasks.

11 References

- [1] H. Dong, S. Zhao, Y. Tian, D. Xiong, S. Xia, M. Zhou, D. Lin, J. Cambronero, J. He, H. Han, and D. Zhang. *SpreadsheetLLM: Encoding spreadsheets for large language models*. In Proceedings of EMNLP, 2024.
- [2] A. Gulati, A. Sen, S. Sarguroh, and R. Paul. *From rows to reasoning: A retrieval-augmented multimodal framework for spreadsheet understanding*. arXiv preprint arXiv:2601.08741, 2025.
- [3] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag. *TabLLM: Few-shot classification of tabular data with large language models*. In Proceedings of AISTATS, 2023.
- [4] G. Li, Z. Du, W. Zheng, and S. Song. *MiMoTable: A multi-scale spreadsheet benchmark with meta operations for table reasoning*. In Proceedings of COLING, 2025.
- [5] Z. Ma, W. Zhang, J. Zhang, C. Yu, T. Zhang, G. Zhang, Y. Luo, J. Wang, and J. Tang. *SpreadsheetBench: Towards challenging real world spreadsheet manipulation*. In NeurIPS Datasets and Benchmarks Track, 2024.

- [6] Y. Sui, M. Zhou, M. Zhou, S. Han, and D. Zhang. *Table meets LLM: Can large language models understand structured table data?* In Proceedings of WSDM, 2024.
- [7] W. Zhao, Z. Hou, S. Wu, Y. Gao, H. Dong, Y. Wan, H. Zhang, Y. Sui, and H. Zhang. *NL2Formula: Generating spreadsheet formulas from natural language queries.* In Findings of EACL, 2024.

12 Appendix A: One-sheet format sample

All samples below encode the same simple worksheet with a header row and one data row.

12.1 Ream

```
#!REAM 9
#!SHEET Revenue
#!HEADERS 1:1
2 | Date | Region | SKU | Qty | UnitPrice | Revenue |
3 | 2024-01-01 | West | SKU-001 | 195 | 49.554 | 9663.05 |
```

12.2 CSV

```
=== Sheet: Revenue ===
Date,Region,SKU,Qty,UnitPrice,Revenue
2024-01-01,West,SKU-001,195,49.554,9663.05
```

12.3 Cell-Address Markdown

```
[Sheet: Revenue]
A2,Date | B2,Region | C2,SKU | D2,Qty | E2,UnitPrice | F2,Revenue
A3,2024-01-01 | B3,West | C3,SKU-001 | D3,195 | E3,49.554 | F3,9663.05
```

13 Appendix B: Suggested next experiments

The current evidence is enough to support Ream as a strong practical choice, but a stronger camera-ready version would add four experiments.

1. **Ablation of addressing and sparsity.** Compare Ream against variants that remove A1 prefixes, remove row labels, or replace R1C1 with A1 formulas while keeping content constant.
2. **Formula-centric tasks.** Evaluate formula tracing, formula generation, and error diagnosis where names and structured references are directly exercised.
3. **Cross-model evaluation.** Replicate the benchmark on at least one non-OpenAI model family.
4. **Item-level significance testing.** Release paired predictions and run McNemar or bootstrap tests for the leading comparisons.